Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Sequential Data and Loops

Prof. Reza Skandari

Today

1. **Recap**
2. Lists and `for` loops
    - Review of prepared material
3. Problem solving and debugging

So far

- Variables allow us to store results
- Conditional statements allow us to make decisions
- While loops let us repeat things
- Algorithms are step-by-step instructions
- Functions let us give names to procedures

Test your instincts

Poll time! Open menti.com and vote on what you think each snippet does.

What is the output sequence?

```
# What is the output?
i = 4
while i > 2:
    print(i*i)
i = i - 1
```

A. 16, 9, 4

B. 16, 9

C. 9, 4

D. An infinite loop

E. I don't know

What is the output?

```
# What is the output?
def fun_function(a, b):
    if b > a:
        return b
    print(a)
x = fun_function(3, 4)
print(x)
```

A. 4

B. 3

C. 3, then 4

D. 4, then 3

E. I don't know

What is the output?

```
# What is the output?
def fun_function(a, b):
    if b > a:
        print(2*b)
    print(a)
x = fun_function(-3, -1)
print(abs(x))
```

A. -2

B. -2, -3, 3

C. -2, -3, Error

D. -2, Error

E. I don't know

Today

1. Recap
2. **Lists and `for` loops**
    - Review of prepared material
3. Problem solving and debugging

While loops

We know how to use while loops:

```
i = 0
while i < 4:
    print(i)
    i = i + 1
```

For loops

We can also use a for loop:

```
# range(n) generates integers 0, ..., n-1
for i in range(4):
    print(i)
```

General form:

```
for item in sequence:
    # do something with item
```

Range:

```
# range(start, end, step)
# step can be negative, for example range(10, 7, -1)
for i in range(5, 11, 2):
    print(i)
```

Lists

So far we have worked with simple data like numbers.

Often we have (ordered) compound data, for example stock prices.

```
prices = [5.2, 4.9, 5.1, 5.6, 7.0]
```

List syntax:

- Square brackets
- Comma-separated
- Values can be a mix of any Python objects, even other lists

List example

Lists are objects: they have properties and methods, like `.append()` below.

```
>>> names = [] # create empty list

>>> names.append('Ruth') # add an item to the end of list
>>> names.append('Mo')

>>> print(names)
['Ruth', 'Mo']
>>> print(len(names))
2
>>> print(names[0]) # indexing: 0, 1, 2, ..., we can also "slice": names[1:2]
Ruth

>>> names[1] = 'Olivia' # change Mo -> Olivia
```

List example: looping

```
>>> for name in names: # convenient to for loop!
>>>     print('Hello ' + name)
Hello Ruth
Hello Olivia

>>> # we can also loop by accessing values at each index:
>>> for index in range(len(names)):
>>>     print('Hi ' + names[index] + ' your index is ' + str(index))
Hi Ruth, your index is 0
Hi Olivia, your index is 1
```

Put it to the test

Poll time! Open menti.com — let's see what for loops actually do.



What is the output sequence?

```
# What is the output?
for i in range(1, 4):
    print(i)
    print(i*i)
```

A. 2, 4, 3, 9, 4, 16

B. 1, 1, 2, 4, 3, 9, 4, 16

C. 1, 1, 2, 4, 3, 9

D. 2, 4, 3, 9

E. I don't know

Are these valid lists?

```
# Are these valid lists?
a = [5, 'hippo', 3]
b = [1]
```

A. Both are

B. Neither is

C. `a` is but `b` is not

D. `b` is but `a` is not

E. I don't know

What is the output?

```
# What is the output?
c = ['Berlin', 'Jakarta', 'Antananarivo']
print(c[0])
c.append('Kinshasa')
print(c[1:3])
```

A. `Berlin`, then `['Jakarta', 'Antananarivo', 'Kinshasa']`

B. `Berlin`, then `['Jakarta', 'Antananarivo']`

C. `Berlin`, then `['Jakarta']`

D. An error

E. I don't know

What is the output?

```
# What is the output?
c = ['Berlin', 'Jakarta', 'Antananarivo']
n = len(c)
for i in range(n):
    print(c[i] + ' to ' + c[i + 1])
```

A. `Berlin to Jakarta`, then `Jakarta to Antananarivo`

B. An error

C. `Berlin to Jakarta`, then `Jakarta to Antananarivo`, then an error

D. `Berlin to Jakarta`, then `Jakarta to Antananarivo`, then `Antananarivo to Berlin`

E. I don't know

What is the output?

```
# What is the output?
c = [['Berlin', 'Germany'], ['Jakarta', 'Indonesia']]
print(c[1])
```

A. `Germany`

B. `['Berlin', 'Germany']`

C. `['Jakarta', 'Indonesia']`

D. An error

E. I don't know

What is the output?

```
# What is the output?
c = [['Berlin', 'Germany'], ['Jakarta', 'Indonesia'], ['Antananarivo', 'Madagascar']]
for element in c:
    print(element[0])
```

A. `Berlin`, `Jakarta`, `Antananarivo`

B. `B`, `G`, `J`, `I`, `A`, `M`

C. `Germany`, `Indonesia`, `Madagascar`

D. An error

E. I don't know

What is the output?

```
# What is the output?
numbers = [1, 2, 3]
more_numbers = numbers
more_numbers[0] = 5
all_numbers = numbers + more_numbers
print(all_numbers)
```

A. [1, 2, 3, 5, 2, 3]

B. [2, 4, 6]

C. [5, 2, 3, 5, 2, 3]

D. [1, 2, 3, 1, 2, 3]

E. I don't know

Python Primitive Data Types

Type	Application	Example	Mutable?
int	Whole numbers	`x = 42`	✗
float	Decimal numbers	`pi = 3.14`	✗
complex	Complex numbers	`z = 2+3j`	✗
bool	True/False values	`flag = True`	✗
str	Text data	`name = "hello"`	✗

Key point: Primitive types are simple and always immutable.

Python Non-Primitive Data Types

Type	Application	Example	Mutable?
list	Ordered collection, allows duplicates	`nums = [1, 2, 3]`	✓
tuple	Ordered, fixed-size collection	`coords = (10, 20)`	✗
set	Unordered, unique values	`items = {1, 2, 3}`	✓
dict	Key-value mapping	`student = {"name": "Ali", "age": 21}`	✓

Key point: Non-primitive types group multiple values; many are mutable.

Careful: passing lists to functions

Lists are mutable, so passing one into a function lets that function change the *original* list, not a copy

```
def add_item(lst):
    lst.append('new')

nums = [1, 2, 3]
add_item(nums)
print(nums) # [1, 2, 3, 'new'] -- the original list got changed!
```

To preserve the original list:

- **Shallow copy**: `b = a[:]`
- **Deep copy** (for lists of lists, etc.): `copy.deepcopy(a)`, from the `copy` package

Returning it doesn't help either

A common instinct: "I'll just return the list and store it under a new name." But if the function never made a copy, you still end up with the *same* list under two names.

```
def process(lst):
    lst[0] = 'updated'
    lst.append('new item')
    return lst

original = ['a', 'b', 'c']
new_list = process(original)

print(new_list) # ['updated', 'b', 'c', 'new item']
print(original) # ['updated', 'b', 'c', 'new item'] -- still changed!
```

`new_list` and `original` are the *same* list object -- assigning the return value to a new variable name doesn't create a copy.

Deep copy example

A shallow copy only copies the outer list -- the inner lists inside are still shared with the original.

```
import copy

matrix = [[1, 2], [3, 4]]

shallow = matrix[:]
shallow[0].append(99)
print(matrix) # [[1, 2, 99], [3, 4]] -- the original changed too!

deep = copy.deepcopy(matrix)
deep[0].append(100)
print(matrix) # [[1, 2, 99], [3, 4]] -- unaffected this time
```

Today

1. Recap
2. Lists and `for` loops
    - Review of prepared material
3. **Problem solving and debugging**

Problem solving

So far, you've mostly worked with exercises that already had tests written for you.

In the real world:

- There are no ready-made tests
- You'll run into bugs and issues on your own
- **Debugging is an essential skill**
- Especially as projects grow larger and more complex

Problem solving

Problem: Given a list containing numbers, write a function that returns a list of cumulative sums up to each element.

1. Make sure you understand the problem. Try examples.
2. Design a solution. Use paper. Find patterns. Check module materials. Try things out. Solve a part of the problem or a special case.
3. Implement the solution. Start from small parts. If your code does not work, trace what the program does line by line to find where the problem is. Use the print function.
4. Test the solution with examples.

Debugging Demo



Review

With lists and for loops, we are ready to start working on a broad range of problems.

Up next:

- Session 3 workshop
- Session 4 prep - computational complexity